In the first session add the default value and while that's being added, you'll run a query in the second session. Running a query simulates an application query running during a structural change.

You'll add the default value two times, a different way each time, demonstrating an important difference between them. The first one is relatively safe and the second one would be likely unsafe for a table with a lot of rows and in a high activity period.

The main difference is that in the first version, the column Default has a static value. This is very fast to add. Run the statement below which adds a static value of "1" to all rows.

sql/migration_safe_modification.sql

```
-- add column with a constant default value
-- This is safe to do and runs fairly quick
ALTER TABLE temp.users ADD COLUMN city_id INTEGER
  DEFAULT 1;
```

After adding the column, run \d temp.users and observe the city_id column and Default value.

Drop the column you just added to prepare for adding it a second time. You can run this from the "psql1" session:

```
ALTER TABLE temp.users DROP COLUMN city_id;
```

Next is the unsafe version. In this version you'll run the ALTER TABLE statement, adding the column in the "psql1" session again.

Run that now.

sql/migration_dangerous_modification.sql

```
-- !!! Dangerous Version !!!
-- Adds a "non-constant" or "volatile" DEFAULT
-- This takes a LOT longer
-- Table is locked in `ACCESS EXCLUSIVE` mode for duration
ALTER TABLE temp.users ADD COLUMN city_id INTEGER
  DEFAULT 1 + FLOOR(RANDOM() * 25);
```

While that's running, try selecting a single row from the same temp.users table in the "psql2" session. Run this SQL query:

```
SELECT * FROM temp.users LIMIT 1;
```

The DDL query in "psql1" might take ten seconds or more to run. The very simple query in "psql2", trying to select one row, will appear "hung" for the whole time until the modification in "psql1" completes, and then the query in "psql2" will finish instantly.